

Knowledge Base Schema

Version: 1.0 Layer: Architecture / Data Schema

This document defines the canonical structure of the Knowledge Base used by the Strategic Knowledge Engine.

The goal of this schema is to ensure:

- structural consistency of the knowledge base
- clear separation of document and chunk metadata
- compatibility with the retrieval architecture
- explainability and traceability of knowledge sources

The schema applies to all artifacts generated by the Knowledge Construction pipeline.

1 Knowledge Base Structure

The Knowledge Base consists of three artifacts generated by the Knowledge Construction pipeline:

knowledge_base/

registry/ document_registry.json

chunks/ chunks.json

vector_index/ embeddings.pt index.faiss

Relationship between artifacts:

Document Registry ↓ doc_id Chunks ↓ index alignment Embeddings ↓ vector mapping Vector Index

The Retrieval Engine uses all three artifacts.

2 Document Registry

The document registry contains metadata describing all documents included in the knowledge base.

Each entry represents one document.

Example:

```
{ "doc_id": "doc_bafin_cloud_outsourcing_2024", "title": "BaFin Guidance on Cloud Outsourcing",  
  "source_file": "bafin_aufsichtsmitteilung_cloud_auslagerung_2024.pdf", "year": 2024,
```

```
"origin": "supervisory_authority", "document_type": "regulatory_guidance",  
  
"knowledge_domain": "insurance_regulation", "content_domain": null,  
  
"vendor": null,  
  
"confidence_level": "high", "confidentiality": "public",  
  
"doc_hash": "sha256_hash_value" }
```

3 Registry Fields

Mandatory fields:

Field	Description
doc_id	stable unique document identifier
title	document title
source_file	original file name
year	publication year
origin	source category (regulator, vendor, advisor, etc.)
document_type	classification of document
knowledge_domain	factual knowledge domain
content_domain	market/analysis domain
confidence_level	trust level of the document
confidentiality	access classification

Optional fields:

Field	Description
vendor	vendor or company referenced in the document
doc_hash	SHA256 hash of the original document

4 Knowledge Domain vs Content Domain

The system distinguishes between two domain types.

Knowledge Domain

Represents factual or normative domains.

Examples:

insurance_regulation insurance_architecture insurance_operations cloud_governance

These sources typically include:

- regulation
- advisory studies
- architecture guidance

Content Domain

Represents market signals, positioning, or narrative content.

Examples:

portfolio_gtm vendor_positioning market_narratives technology_trends

These sources may include:

- marketing material
- blogs
- social media
- opinion pieces

The separation allows the retrieval system to control which types of content influence answers.

5 Chunk Structure

Chunks represent segmented text passages derived from documents.

Each chunk references the source document using doc_id.

Example:

```
{ "chunk_id": "chunk_0123", "doc_id": "doc_bafin_cloud_outsourcing_2024",
```

```
"chunk_index": 12,
```

```
"text": "Cloud outsourcing arrangements must ensure...",
```

```
"tokens": 142,
```

```
"char_start": 4120, "char_end": 4630,
```

```
"metadata": { "section": "Operational Risk", "page": 8 } }
```

6 Chunk Fields

Field	Description
chunk_id	unique identifier for the chunk
doc_id	reference to the source document
chunk_index	sequential position within document
text	text content of the chunk
tokens	token count of the chunk
char_start	character start position in document
char_end	character end position in document
metadata	optional structural information

Optional metadata examples:

section subsection heading page

7 Embeddings and Vector Index

Embeddings represent semantic vector representations of chunks.

embeddings.pt

Contains the embedding vectors in the same order as chunks.json.

Requirement:

embedding[i] corresponds to chunks[i]

index.faiss

FAISS vector index enabling fast similarity search.

The vector index is built from the embedding matrix.

8 Doc ID Generation

Document IDs must be deterministic and human readable.

Recommended format:

doc_<source><topic><year>

Examples:

- doc_bafin_cloud_outsourcing_2024
- doc_dora_regulation_2022
- doc_aws_insurance_cloud_blog_2024

Rules:

- lowercase
- underscore separated
- deterministic

Doc IDs must remain stable across rebuilds of the knowledge base.

9 Design Principles

The schema follows several architectural principles:

Separation of document metadata and chunk metadata

Registry contains document-level metadata. Chunks contain only text-level information.

Immutable Knowledge Base

The knowledge base is generated by the Knowledge Construction pipeline and treated as read-only during runtime.

Explainability

Every retrieved chunk can be traced back to its document and metadata.

Governance

Metadata fields allow retrieval to prioritize reliable sources and downgrade low-confidence material.

10 Relation to Retrieval Architecture

The retrieval system operates on the knowledge base as follows:

Query → candidate chunk retrieval → document grouping via doc_id → governance scoring using registry metadata → document ranking → context selection

The registry therefore plays a central role in retrieval governance.

11 Future Extensions

Possible schema extensions include:

- document versioning
- language metadata
- topic classification
- knowledge graph references

The schema is intentionally minimal to maintain maintainability while supporting future evolution.